

Behavioral Synthesis of ICs

Testability and Dependability

Credentials:

1. Huhn, Sebastian, and Rolf Drechsler. *Design for Testability, Debug and Reliability*. Springer International Publishing, 2021.
2. Zwolinski, Mark. *Digital system design with SystemVerilog*. Pearson Education, 2009.
3. Sahoo, S.S., Das, A., Kumar, A. Fault Tolerant Architectures. In: Chattopadhyay, A. (eds) Handbook of Computer Architecture. Springer, 2023.

Outline

- **Design for Testability**
 - **Fault Models**
 - **Structural Test**
 - **Design for testability**
 - **Built-in self-test**
- **Design for Dependability**
 - **Faults, Errors, and Failures**
 - **Reliability**
 - **Fault Tolerance**
 - **Redundancy**

Design Test

"Testing shows the presence, not the absence of bugs."
--Bertrand Meyer

Introduction

- The high complexity of the manufacturing process of ICs leads inevitably to imperfections or inaccuracies and, hence, to physical defects.
- For instance, such a defect is induced by **process variations**, **surface impurities**, or **contact degradation**. A manufacturer needs to know whether a system (whether at the level of a board, an IC or a whole system) has a defect and therefore does not work in some way.
- While a manufacturer does not want to sell bad systems, equally he or she would not want to reject good systems. Therefore, the need for testing is economic.
- After manufacturing, we will be able to see only the **inputs** and **outputs** of the chip and internal node and pin information is not available, and in such scenarios if the faults are within the chip, then it is very huge loss to the design houses.
- We also need to distinguish between the ideas of **verification**, in which the **design** of a piece of hardware or software is checked, and of **testing**, in which it is assumed that the design is correct but that there may be **manufacturing faults**.

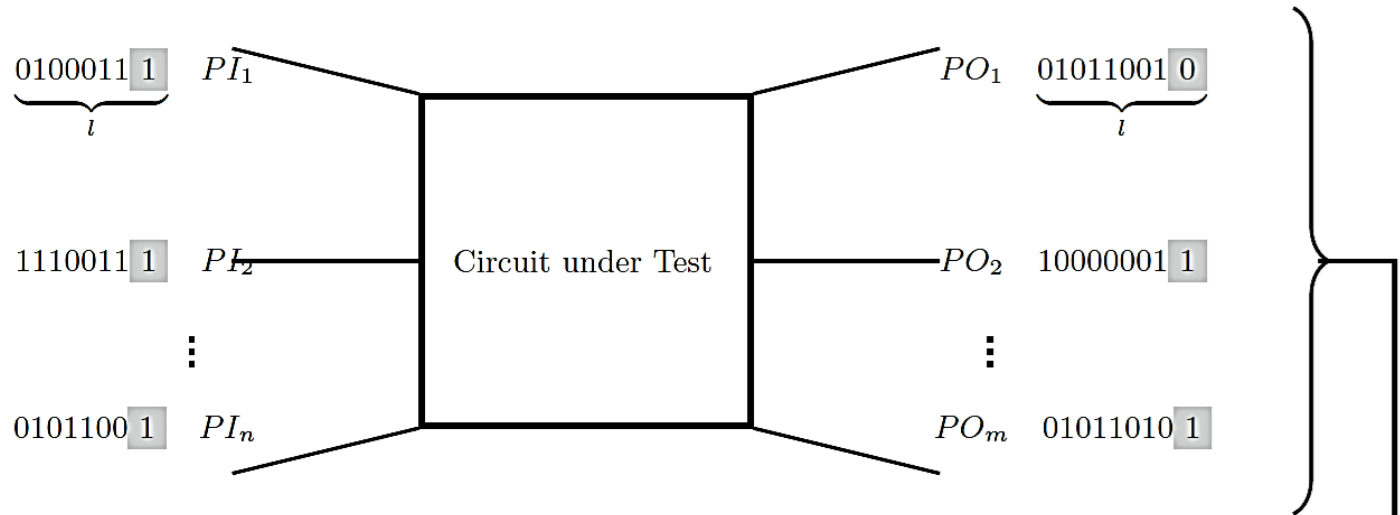
Introduction

- The **circuit test** is inevitably required to prevent the risk of defects potentially harming the intended overall function of the IC. The basic circuit test principle is shown in Figure 1.
- At first, a predetermined logic value is applied in parallel to every of the n PIs of the **Circuit under Test (CuT)**, which forms a **test stimulus**. The circuit test consists typically of a large set (here: l) of test stimuli forming the test set.
- As soon as one test stimulus has been processed by the circuit, the behavior of the CuT is observed at every PO in parallel, which forms, analogously to the test stimulus, the test response.
- During the circuit test, an individual test response is being captured for every test stimuli and, hence, l test stimuli yield l test responses.
- The evaluation of the circuit test is done by comparing the observed test responses with the predetermined (**golden**) ones proving the test result. If the observed

Figure 1. Principal circuit test

test stimuli

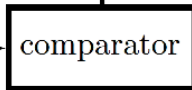
test responses



test result

golden test responses

$\begin{pmatrix} 010110011 \\ 100000011 \\ 010110101 \end{pmatrix}$



Introduction

- There are, in general, two approaches to testing.
 - **Functional Testing**: which is testing whether the system **works correctly or not**. **Functional testing** can imply a long and difficult task because all possible states of a system have to be checked.
 - **Structural Testing**: which is testing whether the system **contains a fault or not**. **Structural testing** is often easier but is dependent upon the exact implementation of a system.
- Note that, the basic principle as shown in Figure 1 remains the same independent of the type of the applied test, although the test generation deviates significantly.
- Furthermore, the resulting **test costs** have to be taken into consideration when generating the test sets. The test costs already hold a large share (about 30%) of the overall costs.
- The **test costs** are directly affected by, for instance, the number of test patterns in the generated test sets and, hence, compact test sets facilitate a test cost reduction.
- Another important aspect concerns the overall **Test Application Time (TAT)** time, particularly, in an environment with limited capacities for a parallel device test like burn-in tests.



Fault Models

“to err is human; to forgive, divine.”,

by Alexander Pope from his poem "*An Essay on Criticism*."

Fault Models

- An electronic system might contain a large number of possible **defects** as a result of the **manufacturing process**.
- For **example**, the printed circuit board could have **breaks** in connections because of bad etching, stress or bad solder joints.
- Equally there may be **short circuits** resulting from the flow of solder.
- The components on a PCB may be at fault – so-called '**population defects**' – caused by having the **wrong components**, wrongly inserted components or **omitted components**.
- The components themselves may fail because the **operating conditions** exceed the component specifications or because of **electromagnetic interference (EMI)** or **heat**.

Fault Models

- Similar defects can occur in integrated circuits. Open circuits may arise from **electromigration** (movement of metal atoms in electromagnetic fields), **current overstress** or corrosion.
- **Silicon or oxide defects**, **mask misalignment**, **impurities** and **gamma radiation** can cause short circuits and incorrect transistor operation. '**Latch-up**', caused by transient currents, forces the output of a CMOS gate to be stuck at a logic value. In **memory** circuits there may be data corruption because of alpha-particles or **EMI**.
- Clearly, to enumerate and check for every possible defect in an electronic system would be an enormous task. Therefore, a distinction is made between **physical defects** and **electrical faults**. The principle of fault modelling is to reduce the number of effects to be tested by considering how defects manifest themselves.
- A **physical defect** will manifest itself as a **logical fault**. This fault may be **static** (e.g. shorts, breaks), **dynamic** (components out of specification, timing failures) or **intermittent** (environmental factors).

Fault Models

- The **relative probabilities of faults** that appear during tests in manufacturing are shown in Figure 2.
- **Dynamic faults** may be further divided into **timing faults** (28%) and **driver faults** (21%).
- **Timing faults** and intermittent faults may be due to poor design. It is difficult to design test strategies for such faults.

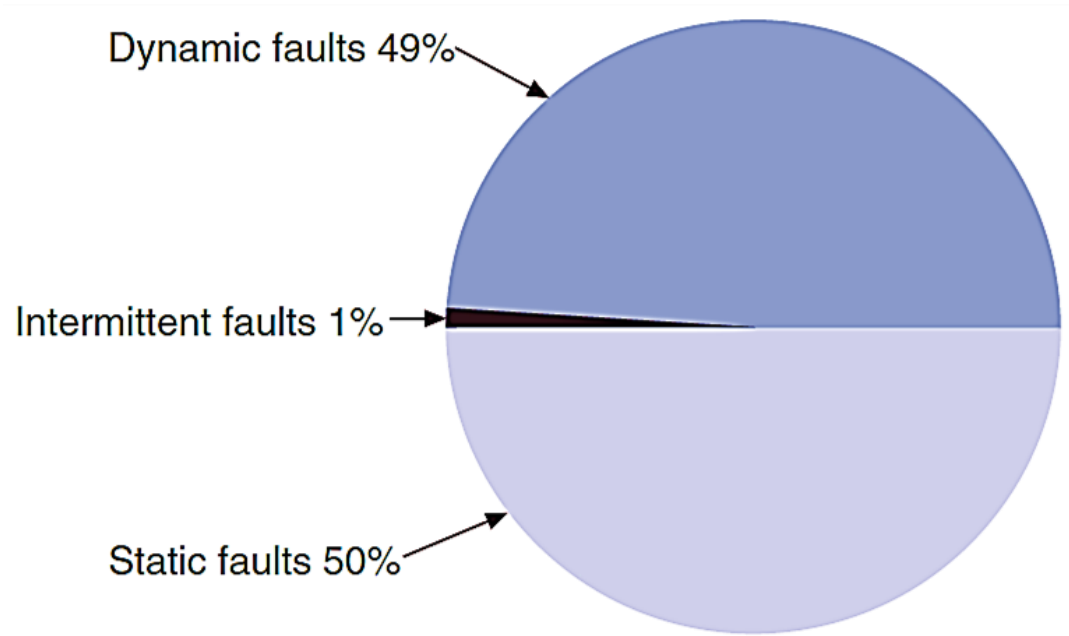


Figure 2: Fault probabilities.

Single-Stuck Fault Models

- **Static faults** are usually modelled by the **stuck fault model**.
- Many physical defects can be modelled as a circuit node being either **stuck at 1 (s-a-1)** or **stuck at 0 (s-a-0)**. Other fault models include **stuck open** and **stuck short** faults. In this approach the overall number of possible fault sites for stuck-at fault within a circuit scales directly with the **number of wires**.
- The **single-stuck fault model (SSFM)** assumes that a fault directly affects **only one node**, and that the node is stuck at either 0 or 1.
- Almost all test pattern generation relies on this model. These assumptions make test pattern generation easier, but the validity of the model is questionable.
- Multiple faults do occur, and multiple faults can theoretically **mask** each other. On the other hand, the model appears to be valid most of the time.
- **Programmable logic** and **memory** have other fault models.

Structural Test

“Structure is more important than perfection.”

Thomas Merton

Structural Test

- The **structural test** of an IC aims at proving the absence of faults, i.e., a set of test stimuli is applied to the CuT, the resulting test responses are observed and compared with the expected ones.
- Every test pattern is meant to **detect at least one fault**. It is assumed that the tested faults are not present in the circuit if the observed test responses equal the expected ones.
- The concept of a **fault** has been introduced as a high level of abstraction of the actual defect and, in particular, their effects on the IC.
- **For instance**, such a defect is about surface impurities leading to certain physical issues during the manufacturing.
- Modeling these physical correlations would require a comprehensive and accurate model, which is not feasible due to a large number of potential defects, and the large size of state-of-the-art IC designs.
- Different fault models have been introduced, which model the effect of defects in the sense of the logical behavior of the CuT.

Structural Test

- Logically, to find a specific stuck-at fault at the point w , the opposite logic value of the stuck-at-fault value at w has to be produced at w . Then, this value has to be **observable** at one of the output PO.
- This means that the value must be **propagated** from the fault site towards a PO, which enables the later comparison against the **golden test response**. Thus, this leads to a three-stage approach for the overall test set generation as follows:
 1. Generate the **list of possible stuck-at fault list** for a given circuit.
 - Typically, different techniques are applied to reduce the number of faults significantly, for example, by taking advantage of **fault collapsing** with respect to the equivalence or the dominance of fault subsets.
 - Perform for every fault the following two steps individually and append the determined test stimuli and responses to the test set:
 2. Determine the necessary test stimuli to justify the opposite value at the fault site, i.e., “1” for s-a-0 and “0” for a s-a-1 fault.
 3. Determine further required test stimuli to sensitize the propagation path from the fault site towards a PO such that the justified value can be observed at a PO.

Fault-oriented test pattern generation

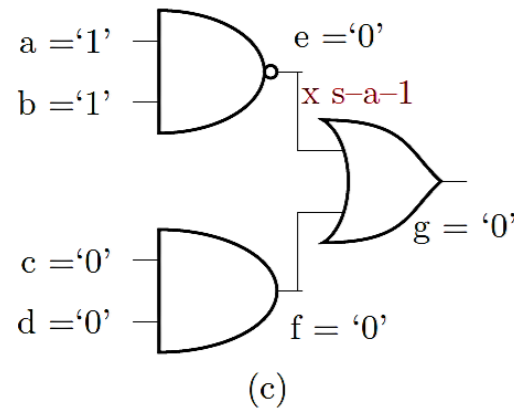
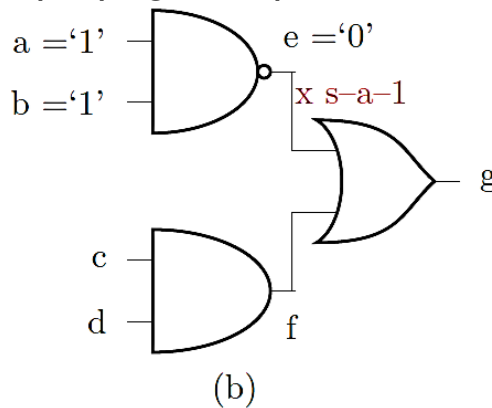
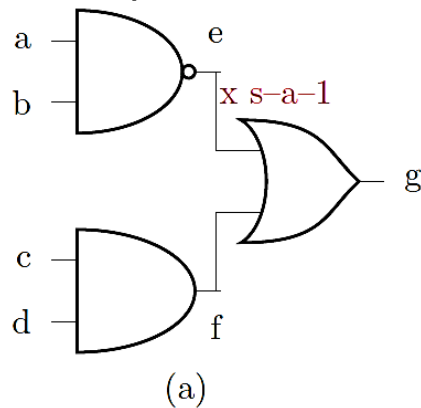
- In theory, a comprehensive testing can be achieved by monitoring the state of every node of the circuit 😊! Obviously, this approach is not practical for integrated circuits.
- We can observe only the outputs of the system and we can control only the inputs of the system.
- Therefore, the task of **test pattern generation** is that of determining a set of inputs to indicate unambiguously whether an internal node is faulty.
- If we consider only **combinational circuits** for the moment, the number of possible input combinations for an n -input circuit is 2^n . We could apply all 2^n inputs (in other words, perform an exhaustive functional test), but in general we want to find the minimum necessary number of input patterns.
- Also, it is possible that, because of the circuit structure, certain faults cannot be detected. Therefore, it is common to talk about the **testability of a circuit**.

Structural Test

- Two important metrics for the structural test concern the **fault and test coverage**, which allow evaluating an existing test set qualitatively.
- **The Fault Coverage (FC)** is defined as: Given a fault list of a circuit, which contains N faults (with respect to a fault model), and a test set. Let the test set detect N_{det} faults out of N , then FC is calculated by $FC = \frac{N_{det}}{N} \times 100\%$.
- **The Test Coverage (TC)** is defined as: Reconsider the test set from above detecting N_{det} faults of a given fault list, then the test coverage is calculated by $TC = \frac{N_{det}}{\hat{N}} \times 100\%$, where $\hat{N}$ represents the number of faults, which are **detectable** when considering the given circuit structure.
- For **example**, redundancies within a given circuit potentially lead to a circumstance where the fault value can never be justified without blocking the propagation path.

Structural Test

→ **Example:** Consider the circuit of Figure 3, which consists of three gates, four inputs and one output. The wire *e* is assumed as the fault site for a s-a-1 fault (Figure 3-a). Moreover, the opposite value, i.e., *e* = “0,” has to be justified. Since that, the PIs of the fan-in cone must be properly assigned with *a* = “1” and *b* = “1” as demonstrated in Fig. 3-b. After the justification, it is required to propagate the value from fault position towards a PO to enable its observability. As shown in Fig. 3c, the two further assignments *c* = “0” and *d* = “0” of the PIs are required to sensitize a propagation path.



Fault-oriented test pattern generation

- Test pattern generation (writing a test) may be **random** or **optimized**.
- One test may cover more than one fault; often faults are **indistinguishable**.
 - If we simply want a **pass/fail test**, once we have found a test for a fault, we can remove faults from further consideration.
 - If we want to **diagnose a fault** (for subsequent repair) we probably want to find all tests for a fault to deduce where the fault occurs.
- The fault cover target may be less than 100%. For large circuits, the time taken to find all possible tests may be excessive.
- Moreover, the higher the cover, the greater the number of tests and hence the cost of applying the test.

Sensitive path algorithm

→ The circuit of Figure 4 has seven nodes, therefore there are 14 stuck faults:

$A/0, A/1, B/0, B/1, C/0, C/1, D/0, D/1, E/0, E/1, F/0, F/1, Z/0, Z/1$

→ where $A/0$ means 'A stuck-at-0', etc.

→ To test for $A/0$, we need to set A to 1 (the fault-free condition – if A were at 0, we would not be able to distinguish the faulty condition from the fault-free state).

→ The presence or otherwise of this fault can be detected only by observing node Z .

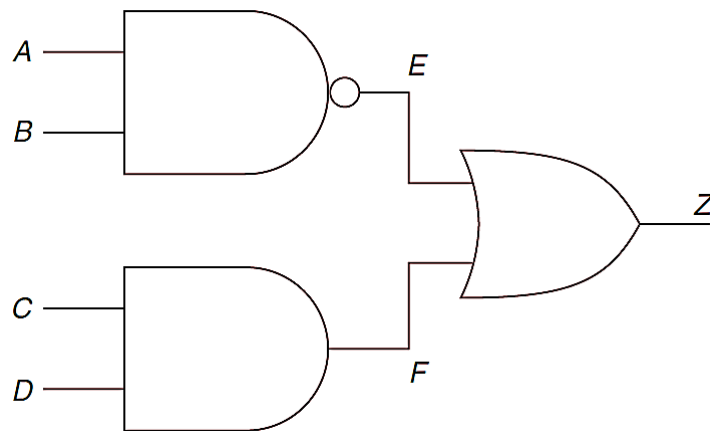
→ We now must determine the states of the other nodes of the circuit that allow the state of A to be deduced from the state of Z .

→ We must establish a sensitive path from A to Z .

Sensitive path algorithm

- **To test for A/0:** A must be set to logical 1
 - If node B is 0, E is 1 irrespective of the state of A. Therefore, B must be set to a logical 1.
 - If F is 1, Z is 1, irrespective of E; hence F must be 0. To force F to 0, either C or D or both must be 0.
 - ❖ If the fault A/0 exists, E is 1 and Z is 1.
 - ❖ If A/0 does not exist, E is 0 and Z is 0.
- We can conclude from this that a test for A/0 is A=1, B=1, C=0, D=1, for which the fault-free output is Z=0.
- This can be expressed as 1101/0.
- Other tests for A/0 are 1110/0 and 1100/0. **Therefore, there is more than one test for the fault A/0.**

Figure 4. Example circuit for test generation.



Sensitive path algorithm

- Let us now consider a test for another fault for the same Example:
 - To test for E/1 requires that $F=0$ to make E visible at Z.
 - Therefore, C or D or both must be 0.
 - To make $E=0$ requires that $A=B=1$.
- So, a test for E/1 is 1101/0. This is the same test as for A/0. 😊
- As we can see one test can cover more than one fault.

Sensitive path algorithm

- The **sensitive path algorithm** therefore consists of the following steps:
 1. Select a fault.
 2. Set up the inputs to force the node to a fixed value.
 3. Set up the inputs to transmit the node value to an output.
 4. Check that the input node values for steps 2 and 3 are consistent.
 5. Check for coverage of other faults.
- The aim is to find the **minimum number of tests that cover all the possible faults**, although 100% fault cover may not be possible.
- Fan-out and reconvergence can cause difficulties for this algorithm.
- Improved algorithms (D-algorithm, PODEM) use similar techniques, but overcome these drawbacks.

Undetectable faults

→ Consider the function

$$Z = A \cdot C + B \cdot \bar{C}$$

→ To avoid hazards, the redundant term may be included, as shown in Figure 5:

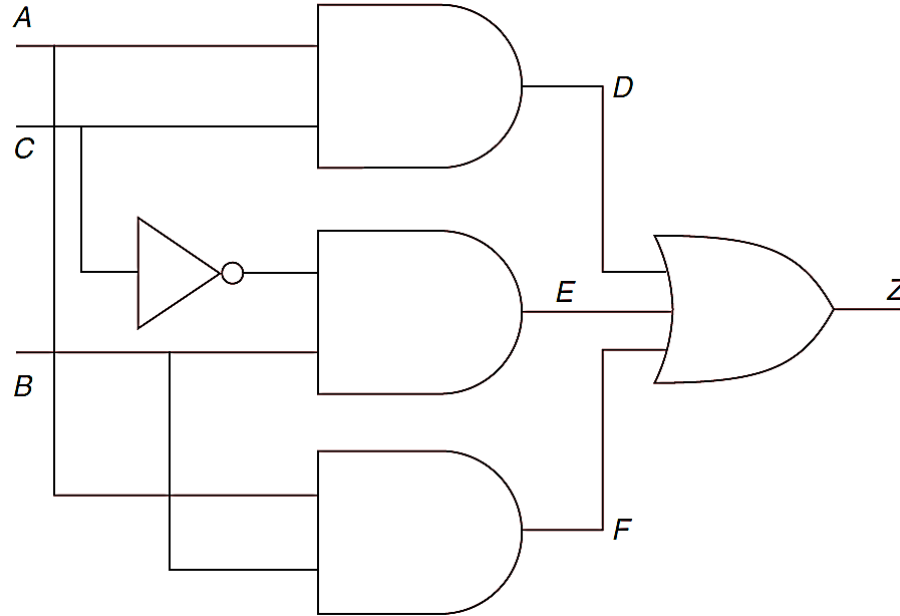
$$Z = A \cdot C + B \cdot \bar{C} + A \cdot B$$

→ We will now try to find a test for F/0.

- This requires that F be set to 1. Hence, $A = B = 1$.
- To transmit the value of F to Z means that $D = E = 0$ (otherwise Z would be 1, irrespective of F).
- For E to be 0, B must be 0 and/or C must be 1.
- Similarly, for D to be 0, A must be 0 and/or C must be 0.
- These three conditions are inconsistent, so no test can be derived for the fault F/0.

Undetectable faults

→ **Figure 5:** Circuit with redundancy.



Undetectable faults

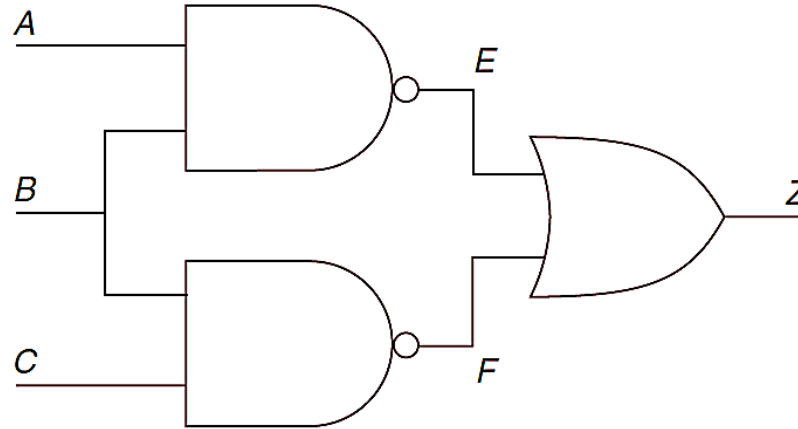
- When there is no test that can be derived to detect a fault, there are three possible responses to this:
 - Either it must be accepted that the circuit is not 100% testable;
 - The redundant gate must be removed, risking a hazard;
 - The circuit must be modified to provide a control input for testing purposes, to force D to 0 when $A = C = 1$.
- In general, untestable faults are due to **redundancy**.
- Conversely, redundancy in combinational circuits will mean that those circuits are not fully testable.

The D-algorithm

- The simple **sensitized path procedure** does not handle reconvergent paths adequately.
- For **example**, consider the circuit of Figure 6. To find a test for B/1 requires that B be set to 0.
 - To propagate the state of B to E requires that A is 1, and to propagate E to Z requires that F is 0.
 - The only way that F can be at 0 is if B and C are both 1, but this is not the case when B = 0.
 - Apparently, therefore, the sensitive path algorithm cannot find a test for B/1.
- In fact, 101/1 is a suitable test, because under fault-free conditions E, F and Z are all at logical 1; when B/1, all three nodes are at logical 0.

The D-algorithm

→ **Figure 6:** Example circuit for D-algorithm.



The D-algorithm

- The **D-algorithm** overcomes this problem by introducing a five-valued algebra: $\{0, 1, D, \bar{D}, X\}$
 - D represents a node that is logical 1 under fault-free (normal) conditions and logical 0 under faulty conditions.
 - $\bar{D}$ represents a normal 0, and a faulty 1.
 - X is an unknown value.
- The values of D and $\bar{D}$ are used to represent the state of a node where there is a fault and also the state of any other nodes affected by the fault.
- The **D-algorithm** works in the same way as the sensitive path algorithm, however, if step 4 fails, the algorithm backtracks.
- In both steps 2 and 3 it is possible that more than one combination of inputs generates the required node values.
- If necessary, all possible combinations of inputs are examined.

The D-algorithm

- Table 1 shows the inputs required to establish a fault at an internal node, to transmit that fault to an output, and to generate a fixed value (to establish or propagate a fault).
- It shows how fault conditions can reconverge.
- In all cases, the inputs A and B are interchangeable.
- The table can be extended to gates with three or more inputs.
- The symbol '—' represents a 'don't care' input.

The D-algorithm

Table 1: Truth tables for the D-algorithm.

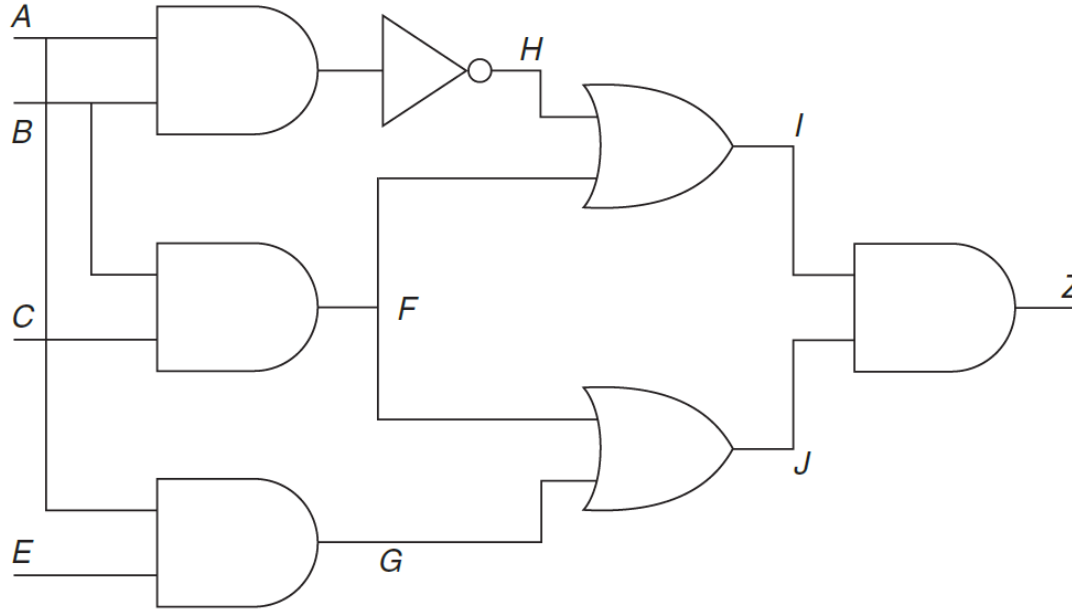
	[AND]			[OR]			[NAND]			[NOR]			[NOT]	
	A	B	Z	A	B	Z	A	B	Z	A	B	Z	A	Z
Establish fault-sensitive condition	1	1	D	0	0	$\bar{D}$	1	1	$\bar{D}$	0	0	D	1	$\bar{D}$
	0	–	$\bar{D}$	1	–	D	0	–	D	1	–	$\bar{D}$	0	D
Transmit fault	D	1	D	D	0	D	D	1	$\bar{D}$	D	0	$\bar{D}$	D	$\bar{D}$
	$\bar{D}$	1	$\bar{D}$	$\bar{D}$	0	$\bar{D}$	$\bar{D}$	1	D	$\bar{D}$	0	D	$\bar{D}$	D
Generate fixed value	1	1	1	1	–	1	1	1	0	1	–	0	1	0
	0	–	0	0	0	0	0	–	1	0	0	1	0	1
Reconvergence	D	D	D	D	D	D	D	D	$\bar{D}$	D	D	$\bar{D}$		
	$\bar{D}$	$\bar{D}$	$\bar{D}$	$\bar{D}$	$\bar{D}$	$\bar{D}$	$\bar{D}$	$\bar{D}$	D	$\bar{D}$	$\bar{D}$	D		
	D	$\bar{D}$	0	D	$\bar{D}$	1	D	$\bar{D}$	1	D	$\bar{D}$	0		

The D-algorithm

- To see how the D-notation can be used, consider the circuit of Figure 7.
- To test for $A/0$, node A is first given a value D , which can be propagated via node H or via node G.
 - To propagate the D to node H, node B must be 1. Node H then has the value $\bar{D}$.
 - To propagate $\bar{D}$ this to 1 requires F to be 0, and to propagate the value to Z means J must be 1.
 - If F is 0 and J is 1, G must be 1; therefore, nodes A and E must both be 1. At this point we hit an inconsistency as node A has the value D . We have to return to the last decision made, which in this case was the decision to propagate the value of A through to H.
 - The alternative is to propagate the D at A to G. Thus, E must be 1; to propagate the value to J, F must be 0, and to propagate to Z, 1 must be 1.
 - Hence H must be 1. As A is already assigned, B must be 0. This is consistent with F being 0 and C may be either 1 or 0.

The D-algorithm

→ **Figure 7:** Example circuit for D-algorithm.



The D-algorithm

- The D-algorithm as presented here requires further refinement before it can be implemented as an EDA program.
- In particular the rules for detecting inconsistencies require more detail.
- Table 2 shows what happens when two fault-free or faulty values are propagated by different routes to the same node.

Table 2 Intersection rules for the D-algorithm.

$\cap$	0	1	X	D	$\overline{D}$
0	0	ϕ	0	ψ	ψ
1	ϕ	1	1	ψ	ψ
X	0	1	X	D	$\overline{D}$
D	ψ	ψ	D	μ	λ
$\overline{D}$	ψ	ψ	$\overline{D}$	λ	μ

ϕ inconsistent logic values

ψ inconsistency between logic values and fault values

μ allowed intersection between fault values

λ inconsistent fault value

The D-algorithm

- The **D-algorithm** is an algorithm in the true sense of the word – if a solution exists, the D-algorithm will find it.
- The search for a solution can, however, be very **time-consuming**.
- If necessary, every possible combination of node values will be examined.
- Subsequent test pattern generation algorithms have attempted to speed up the D-algorithm by improving the decision-making within the algorithm.
- Examples include 9-V, which uses a nine-valued algebra, and PODEM.

Design for Testability

Testability is a primary driver of quality.

Design for testability

- **Testability** can be a somewhat abstract concept.
- One useful definition of testability breaks the problem into two parts:
 - **Controllability** – the ability to control the logic value of an internal node from a primary input.
 - ❖ The questions is: can we control all the nodes to establish whether there is a fault?
 - **Observability** – the ability to observe the logic value of an internal node at a primary output.
 - ❖ The question is: can we observe and distinguish between the behavior of a faulty node and that of a fault-free node?

Design for testability

- Compared with **combinational circuits**, testing of **sequential circuits** is much more difficult because the **current state** of the circuit as well as its inputs and outputs must be taken into account.
- Although in many cases it is possible, at least in theory, to derive tests for large, complex, sequential circuits, in practice it is often easier to modify the design to increase its testability.
- In other words, extra inputs and outputs are included to increase the **controllability** and **observability** of internal nodes.
- Testability can be enhanced by **ad hoc design guidelines** or by a **structured design methodology**.

Ad hoc testability improvements

- If one of the objectives of a design is to **enhance the testability of that design**, there are a number of styles of design that should be avoided, including:
- **Redundant logic**. As seen in the previous discussions, redundant combinational logic will result in potentially **undetectable faults**. This means that the design is not fully testable and also that time may be spent attempting to generate tests for these undetectable faults.
 - **Asynchronous sequential** systems (and in particular unstructured asynchronous systems) are difficult to synchronize with a tester. The operation of a synchronous system can be halted with the clock. An asynchronous system is, generally, uncontrollable. If asynchronous design is absolutely necessary, confine it to independent blocks.
 - **Monostables** are sometimes used for generating delays. They are extremely difficult to control, and again should be avoided.

Ad hoc testability improvements

- On the other hand, there are a number of **modifications** that could be made to circuits to **enhance testability**.
- The single most important of these is the **inclusion of some form of initialization**. A test sequence for a sequential circuit must start from a **known state**. Therefore, initialization must be provided for all sequential elements. Any defined state will do – not necessarily all zeros. Multiple initial states can be useful.
- The cost of enhancing testability includes that of **extra I/O pins** (including interfaces, etc.), **extra components** (MUXs), **extra wiring**, and the degradation of performance because of **extra gates in signal paths**; in general, there are more things to go wrong.
- Against this must be set the benefit that the circuit will be easier to test and hence the manufacturer and consumer can be much more confident that working devices are being sold.

Structured design for test

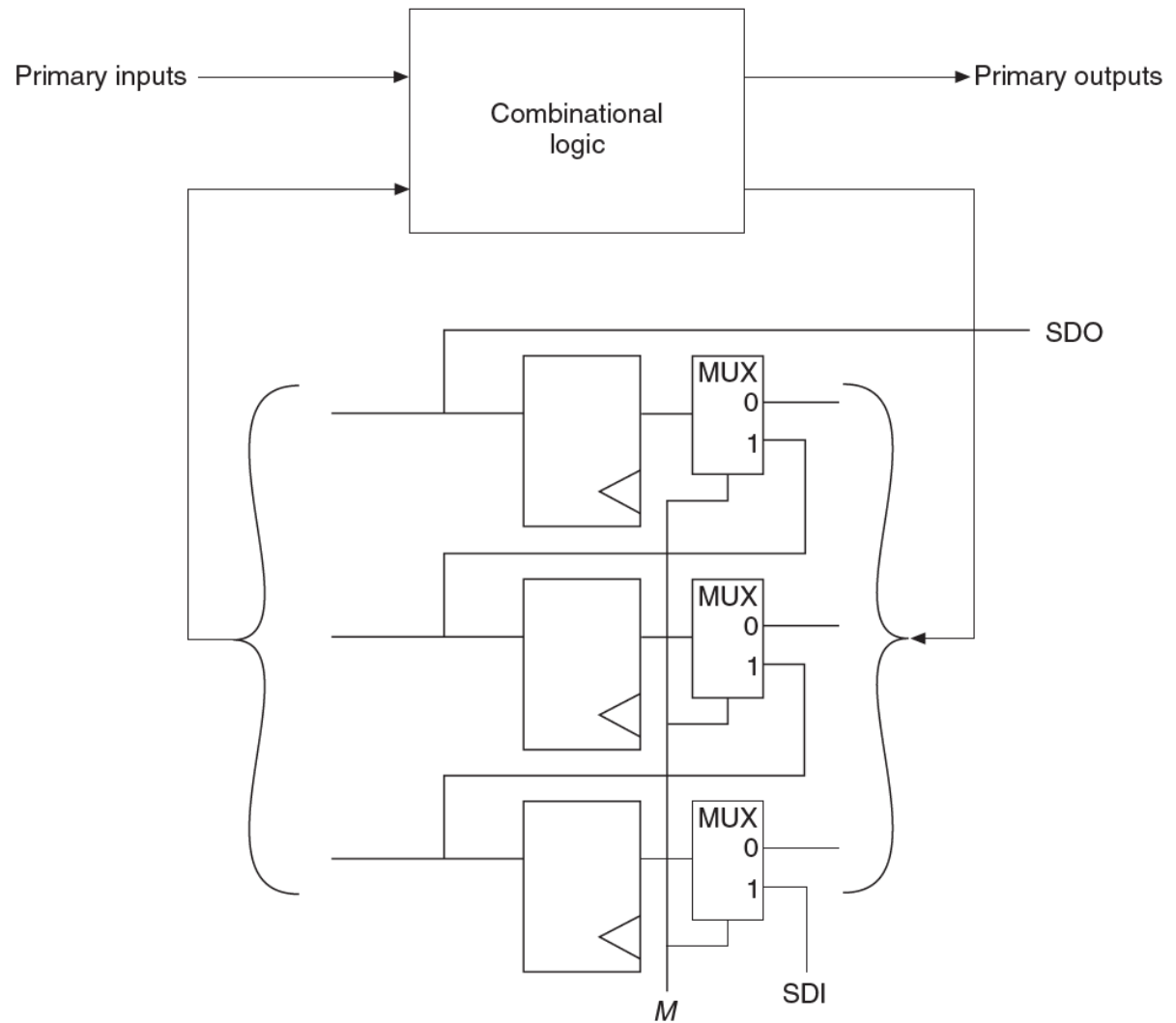
- The techniques described in the previous slides are all enhancements that can be made to a circuit after it has been designed.
- A structured design for test method should consider the testability problem from the beginning.
- Testing **combinational circuits** is **relatively easy** provided there is no redundancy in the circuit. The number of test vectors is (much) less than $2^{(\text{no. of inputs})}$.
- Testing **sequential circuits** is **difficult** because such circuits have **states**. A test may require a **long sequence of inputs** to reach a particular state.
- **Some faults may be untestable**, because certain states cannot be reached.
- Synchronous sequential systems, however, can be thought of as combinational logic (next state and output logic) and sequential logic (registers). Therefore, the key to structured design for test is to separate these two elements.

Structured design for test

- A **synchronous sequential system** does not, however, provide direct control of all **inputs to the combinational logic**, does not allow **direct observation** of all **outputs** from the combinational logic, and does not allow **direct control** or **observation** of the **state variables**.
- The **scan-in, scan-out** (SISO) principle overcomes these problems by making the state variables directly accessible by connecting all the **state registers as a shift register**, for test purposes, as shown in Figure 8.
- This **shift register** has a mode control input, M.
 - In normal, operational mode, M is set to 0.
 - In scan mode, M is set to 1 and the flip-flops form a shift register, the input to the shift register being the scan data in (SDI) pin and the output being the scan data out (SDO) pin.

Structured design for test

Figure 8: SISO principle.



Structured design for test

- If the **combinational logic** has no redundancies, a set of test patterns can be generated for it, as if it were **isolated from the state registers**.
- The test patterns and the expected responses then have to be sorted because this test data is applied through the primary inputs and through the state registers using the scan path.
- Similarly, the outputs of the combinational logic are observed through the primary outputs and using the scan path.

Structured design for test

→ The scan path is used to test a sequential circuit using the following procedure.

1. Set $M = 1$ and test the flip-flops as a shift register.
 - If a sequence of 1s and 0s is fed into SDI, we would expect the same sequence to emerge from SDO delayed by the number of clock cycles equal to the length of the shift register (n).
 - A useful test sequence would be 00110 . . . which tests all transitions and whether the flip-flops are stable.
2. Test the combinational logic.
 - a) Set $M = 1$ to set the state of the flip-flops after n clock cycles by shifting a pattern in through SDI.
 - b) Set $M = 0$. Set up the primary inputs. Collect the values of the primary outputs. Apply one clock cycle to load the state outputs into the flip-flops.
 - c) Set $M = 1$ to shift the flip-flop contents to SDO after $n - 1$ clock cycles.

Structured design for test

- Note that step 2(a) for the next test can be done simultaneously with step 2(c) for the present test.
- In other words, while the contents of the shift register are being shifted out, new data can be shifted in behind it.
- The **benefit** of using a **scan path** is that it provides an easy means of **making a sequential circuit testable**.
- If there is no redundancy in the combinational logic, the circuit is fully testable.
- The problem of test pattern generation is reduced to generating tests only for the combinational logic.
- This can mean that the time to test one device can be greater than would be the case if specific sequential tests had been generated.

Structured design for test

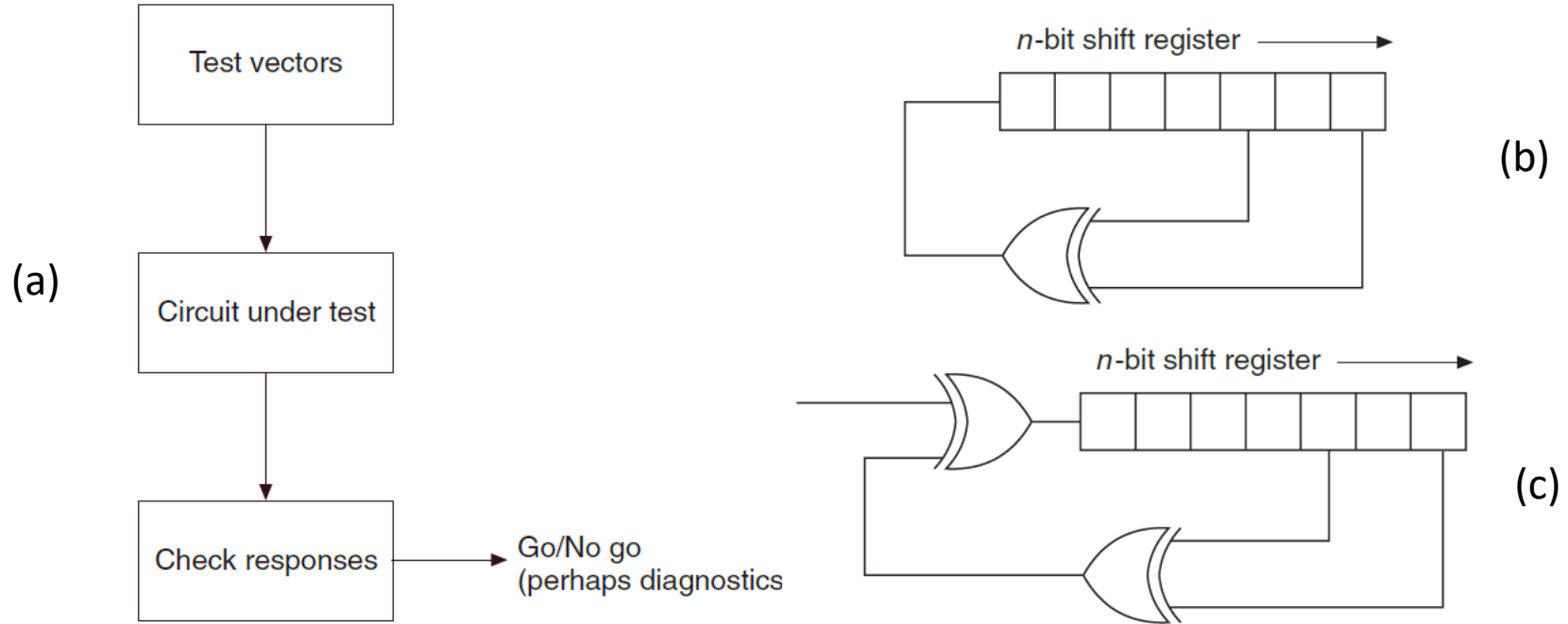
- The **costs of SISO include extra hardware**: at least one extra pin for M; SDI and SDO can be shared with other system functions by using multiplexers.
- An **extra multiplexer** is needed for each flip-flop and **extra wiring** is needed for the scan path. Hence this can lead to **performance degradation** as the delay through the next state logic is increased.
- To minimize the wiring, it makes sense to decide **the order of registers** in the scan path after placement of devices on an ASIC or FPGA has been completed. The order of registers is unimportant provided it is known to the tester.
- SISO is relatively well accepted as a design methodology. Most VLSI circuits include some form of scan path, although this is not usually documented.
- A number of variations to SISO have been proposed including multiple scan paths – put flip-flops in more than one scan path to shorten the length of each path and to shorten the test time – and partial scan paths, whereby some flip-flops are excluded from the scan path.

Built-in self-test

- As with all testing matters, the motivation for **built-in self-test** (BIST or BIT for builtin test) is economic.
- The inclusion of structures on an integrated circuit or board that not only enhance the testability but also perform some of the testing simplifies the test equipment and hence reduces the cost of that equipment.
- BIST can also **simplify test pattern generation** because the test vectors are generated internally, and it allows field testing to be performed for perhaps years after manufacture.
- Overall, therefore, BIST should increase user confidence. The principle of BIST is shown in Figure 9.a.
- The test vector generation and checking are built on the same integrated circuit as the circuit under test. Thus there are two obvious problems: **how to generate the test vectors** and **how to check the responses**.
- It would, in principle, be possible to **store pre-generated vectors in ROM**. There could, however, be a very **large number of vectors**. Similarly, it would be possible to have a **look-up table of responses**.

Built-in self-test

→ **Figure 9:** (a) BIST principle, (b) Linear Feedback Shift Register, (c) Single-Input Signature register



Built-in self-test

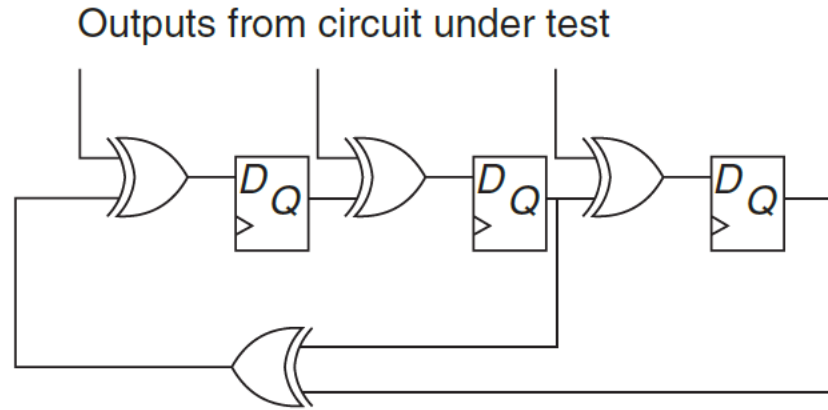
- If an **exhaustive test** were conducted, all possible test vectors could be generated using a **binary counter**.
- This could require a substantial amount of extra combinational logic. A simpler solution is to use a **linear feedback shift register (LFSR)**, which is a **pseudo-random number generator** that generates all possible states (except the all 0s state) but requires less hardware than a binary counter, as shown in Figure 9.b.
- A similar structure can be used instead of a **look-up table** to collect the responses. The **single-input signature register (SISR)** is shown in Figure 9.c.
- This is an LFSR with a single data input. The register holds the **residue from a modulo-2 division**.
 - In other words, it compresses the stream of input data to produce a signature that may be compared, after a certain number of cycles, with a known **good signature**.

Built-in self-test

- Another variant is the **multiple input signature register (MISR)**, shown in Figure 10. Again, this is a modified LFSR but with more than one data input. Thus, a number of output vectors can be gathered and compressed.
- After a **number of clock cycles the signature** in the register should be unique.
- If the circuit contains a **fault**, the register should contain an **incorrect signature**, which can easily be checked.
- This approach will obviously fail if the MISR is sensitive to errors.
- The probability that a faulty circuit generates a correct signature tends to for an n-stage register and long test sequences.

Built-in self-test

→ **Figure 10:** Multiple Input Signature Register (MISR).

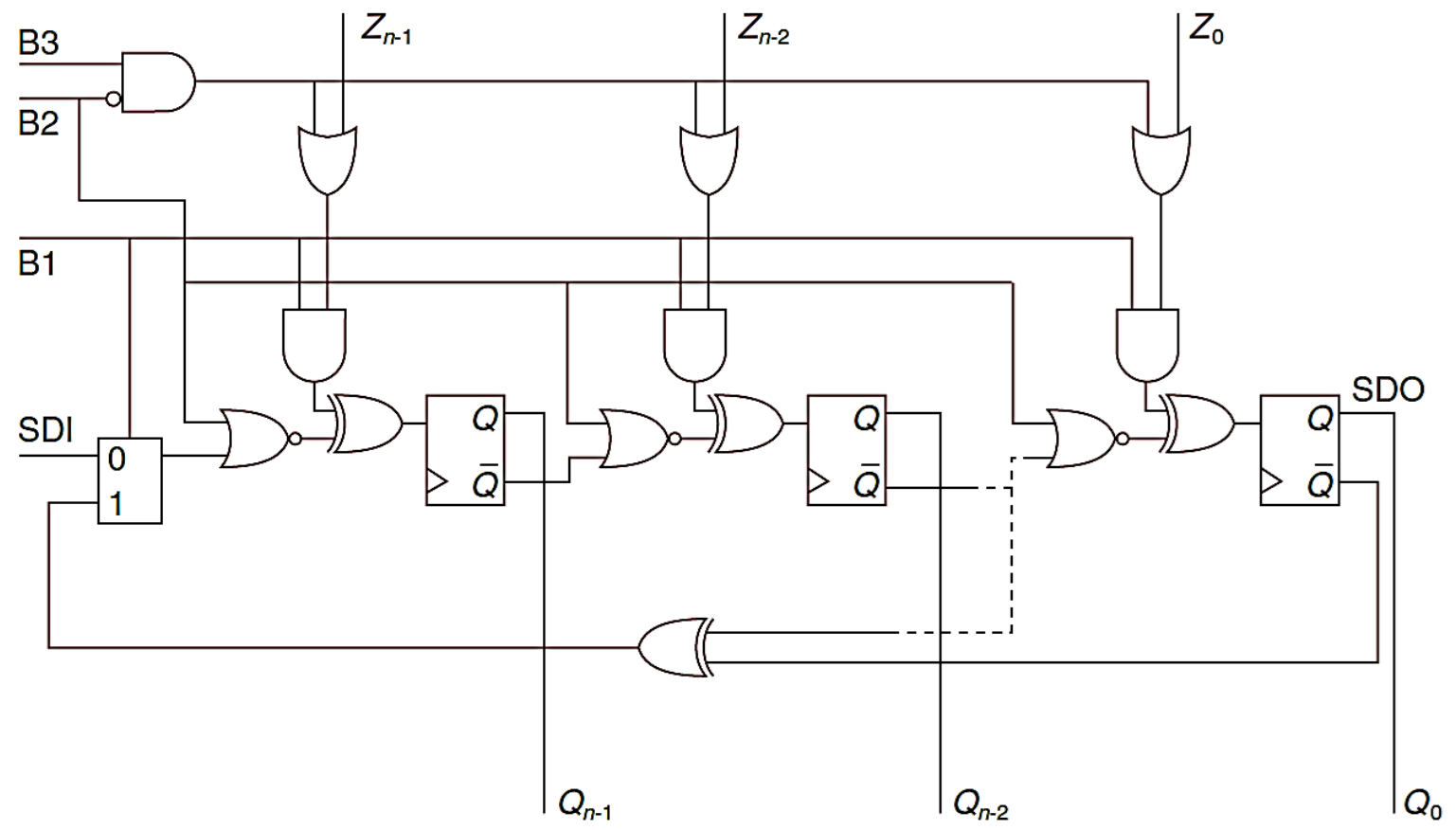


Built-in logic block observation (BILBO)

- The LFSR and MISR, described above, are specialist logic blocks.
- To include BIST in a circuit using such blocks would require **additional registers** to those required for normal operation.
- A scan path reuses the existing registers in a design for testing; in much the same way, **built-in logic block observation (BILBO)** registers are used both for normal operation and for BIST.
- A typical BILBO architecture is shown in Figure 11, where three control signals are required, which control the circuit as shown in the table.

B1	B2	B3	Mode
1	1	–	Normal
0	1	–	Reset
1	0	0	Signature analysis MISR
1	0	1	Test pattern generation LFSR
0	0	–	Scan

Built-in logic block observation (BILBO)



Design for Dependability

In the architecture of resilience, dependability is not an afterthought; it is the cornerstone. Design for Dependability, where every line, curve, and code becomes a pledge to durability and trust.



Fault-Tolerant System Design

- **Fault-tolerant computing** has been the cornerstone of reliable computing using electronic systems.
- **Traditionally**, fault-tolerant system design has been primarily driven by the system's operating environment and the resulting fault scenarios due to **external disturbances**.
- However, with the **growing unreliability** of modern semiconductor technologies, reliable computing requires designing fault tolerance across multiple layers of the system stack.
- Further, the growing impact of intrinsic fault mechanisms such as **aging** requires designing fault-tolerant architectures across application domains.
- To this end, **emerging technologies**, both semiconductor devices and applications, have brought about novel challenges to designing fault-tolerant architectures.

Fault-Tolerant System Design

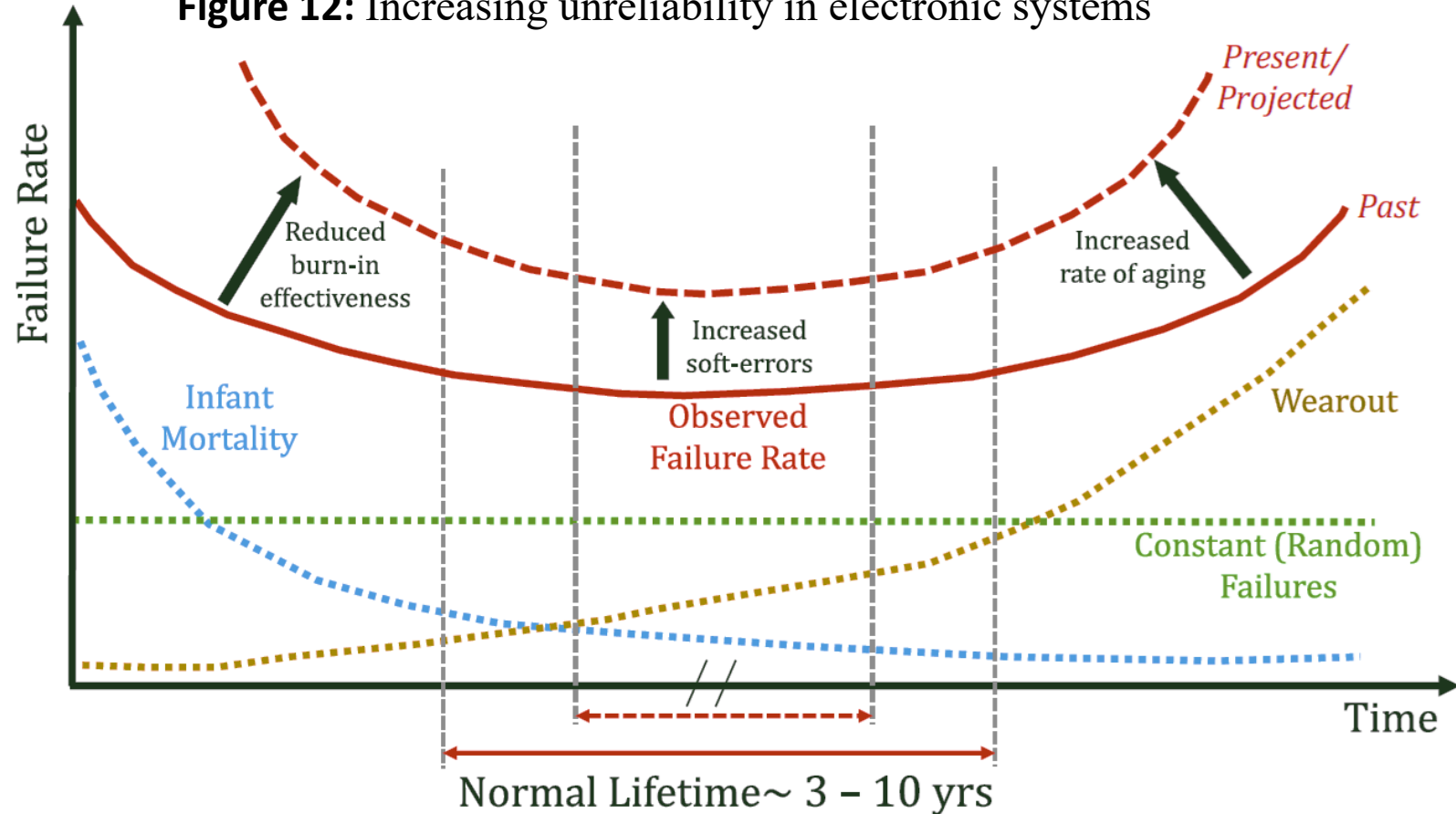
- From deciphering Lorenz-encrypted messages to rendering of virtual environments for the metaverse, computing systems have witnessed **exponential growth** in both the **quantity** and **variety** of **computation workloads**.
- **Moore's Law** (Moore 2006) has been one of the major driving forces enabling electronic systems to cater to such increasing computation demands.
- Such growth has been possible primarily due to the **aggressive semiconductor technology scaling**.
- Technology scaling, along with architectural innovations has also led to increasing **operating frequency**, at least until 2005. However, the reducing **transistor sizes** along with the **increasing power density** (owing to higher clock frequency) have also resulted in **increasing manufacturing defects** and **reduced robustness to external faults**.
- Further, continued efforts toward **supply voltage scaling**, in order to **reduce power density**, have exacerbated the effects of **external disturbances**.

Fault-Tolerant System Design

- While the **logical masking** of **externally induced soft errors** has remained unchanged, the **electrical masking** and **latching window masking** have reduced due to the **supply voltage scaling** and **higher clock frequencies**, respectively.
- This **increasing unreliability** of electronic systems is captured in the **bathtub curve**, shown in Figure 12, showing the effect of **increasing faults** in the three stages of typical electronic systems' life cycle, namely:
 - **Infant mortality**, primarily caused by premature failure of weak components as a result of manufacturing defects and burn-in testing and
 - **Constant failures** due to random faults and
 - **Wear-out based** faults due to aging.

Fault-Tolerant System Design

Figure 12: Increasing unreliability in electronic systems



Fault-Tolerant System Design

- The solid curve in the figure shows the net effect of all three factors.
- With aggressive technology scaling, the rate of **manufacturing defects** has increased, resulting in higher **infant mortality** and higher susceptibility to **aging related faults**.
- As mentioned earlier, the higher **soft error** rate results in increasing **constant failure rate** due to **random faults**.
- Further, the **increasing power density** due to **higher clock speeds** and **parallel processing** (in multicore systems, pipelines) results in **accelerated aging**.
- The net result of all these factors is the increased failure rate as shown by the dashed bathtub curve in the figure 12.

Fault-Tolerant System Design

- The effect of **increasing faults** can be observed across all types of **architecture components** – **computation**, **communication** and **storage** – resulting in the degradation of different **Quality of Service (QoS)** metrics of an application.
- If unmasked, **soft errors** in memory elements and interconnects result in reduced **functional reliability**.
- Similarly, **degradation** in interconnects and logic elements may lead to slower components leading to **reduced timing reliability**.
- **Lifetime reliability** of electronic systems is adversely affected by both **aging** and **permanent faults** of the components.

Fault-Tolerant System Design

- **Redundancy** has been the primary approach for achieving **fault tolerance** across the computation stack. It usually involves replicating execution across
 - 1) multiple components (spatial),
 - 2) a single component (temporal), and
 - 3) additional data (information) to detect and/or mask faults.
- The **implementation cost** and **efficacy** of each type of redundancy may vary for different applications, and given the **system's constraints**, a subset of such methods may prove to be infeasible for the application.
- Further, depending upon the **application domain**, a system may prioritize one/more among functional, timing, and lifetime **reliability**.

Fault-Tolerant System Design

- For **instance**, in real-time systems, the **timeliness of execution** has the highest priority.
- Similarly, in financial and scientific computations, the **accuracy** of calculations would be more important than the execution time.
- Additionally, in systems such as consumer products and space missions, **extended lifetime** of the system may have higher priority.
- Also, in a system executing **multiple applications**, each application may have varying criticality with reference to each reliability-related performance metric.
- In this scenario, a uniform approach to fault tolerance can result in under-/overdesign.
- Therefore, designing **fault-tolerant** architectures forms an important aspect of **application-specific computing**.

Faults, Errors, and Failures

- Faults show us what needs attention.
- Errors show us what needs fixing.
- Failures show us what needs improvement.

Faults, Errors, and Failures

- The events in a system related to fault tolerance can be classified as one of
 - Failure,
 - Error,
 - Fault.
- An application **failure** refers to an event where the service delivered by the system deviates from the expected service defined by the application requirements.
- An **error** refers to the deviation of the system from a correct service state to an erroneous one.
- **Faults** refer to the adjudged or hypothesized cause of the error. The faults in any computing system may be caused by either **physical faults** affecting the hardware or due to imperfect **software implementation**. Here we limit the discussion to physical faults only.

Faults, Errors, and Failures

- A major **classification of faults** is based on the **frequency** and **persistence** of the occurrence of the faults:
- **Transient Faults** occur at a particular time, remain in the system for some period, and then disappear. Such faults are initially dormant but can become active at any time. Examples of such faults occur in hardware components that have an adverse reaction to some external interference, such as electrical fields or radioactivity.
- **Intermittent Faults** show up in systems from time to time due to some inherent design issue or aging. An example is a hardware component that is heat-sensitive – it works for some time, stops working, cools down, and then may start to work normally again.
- **Permanent Faults** such as a broken wire or a software design error show a more persistent behavior than intermittent faults – start at a particular time and remain in the system until they are repaired.

Fault Masking

- The **fault mechanisms** can result in **bit flips** in memory and/or in some logic node. However, not all faults may lead to **errors** or **failures**.
- Faults can eventually lead to application failure only if they are not **masked** at any stage.
- The **masking of faults** (and errors) can be attributed to one or more of the following phenomena:
 - **Logical Masking**: The inputs to the combinational circuit or computation do not allow a logical path between the node affected by the **Single Event Upset (SEU)** and the output.

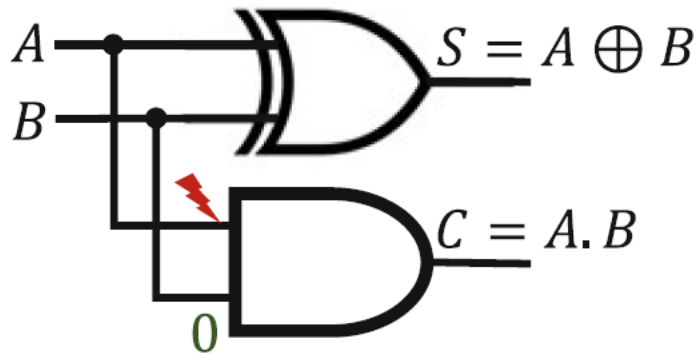
As shown in Figure 13.a, in a half-adder circuit, any incorrect logic value at one of the inputs of the “and” gate does not impact the carry-out value as long as the other input is at logical 0.

Such masking effects are usually not affected by technology scaling and are **application-/design-specific**.

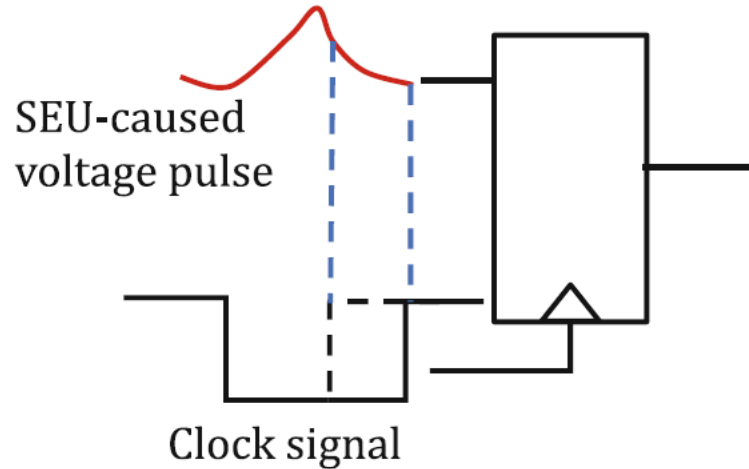
Fault Masking

→ **Figure 13:** Fault Masking:

- a) Logical masking in half-adder combinational circuit.
- b) Reduced latching-window masking due to faster clocks



(a)



(b)

Fault Masking

- **Electrical Masking**: The changed pulse due to an SEU is attenuated during propagation and is too small to be captured.

Continuous voltage scaling in advanced technology models has led to decreasing Q_{crit} in a high energy particle impact and higher susceptibility to transient faults by external charged particles, leading to higher **Soft Error Rate (SER)**.

- **Latching-window Masking**: The affected pulse arrives too early or too late to be captured by the storage element.

As shown in Figure 13.b, deeper pipelines and the resulting higher clock frequencies tend to increase the probability of the incorrect logic value being latched by the storage elements.

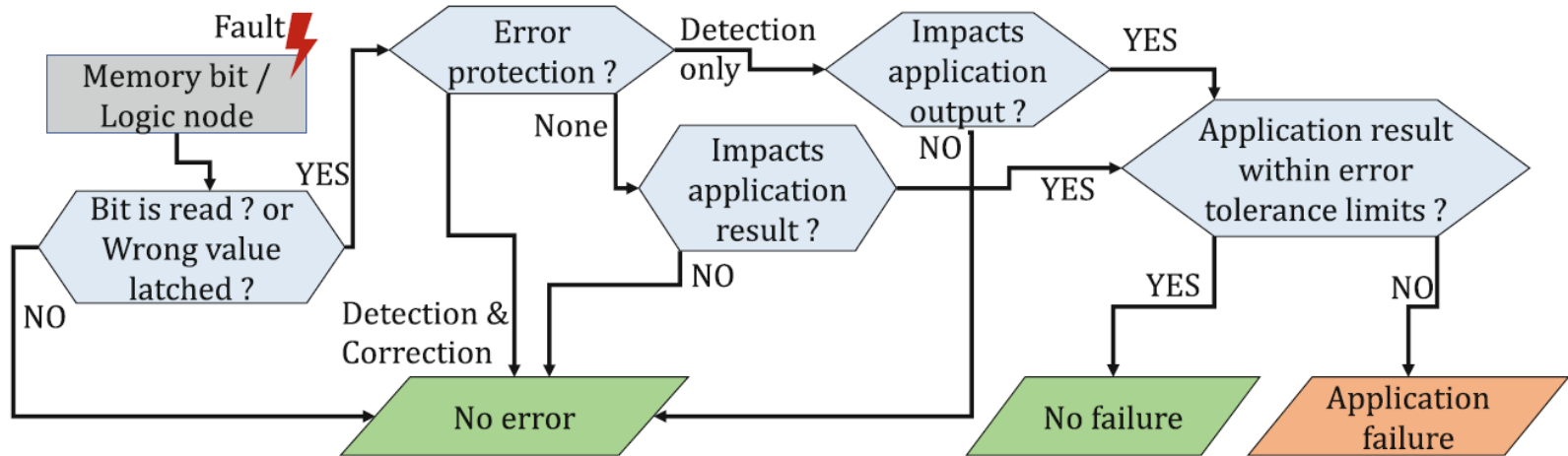
Hence, there has been a reduction in the latching window masking of transient faults, leading to higher SER.

Fault Masking

- The fault masking described above refers to more **implicit** phenomenon that may occur in a system.
- Additional **fault mitigation methods** are usually implemented at multiple abstractions for improving the fault tolerance of the system.
- Given the implicit fault masking and fault mitigation methods, Figure 14 shows the different scenarios that can occur as a result of a hardware fault.
- As seen in the figure, an **error** occurs in the system only when a **wrong logic value** is either read from a storage bit or latched from a gate output and not mitigated by any **error protection mechanisms**.
- Similarly, an **application failure** occurs only when the **error** is not masked by the application logic and the resulting deviation is beyond the application's tolerance limits.

Fault Masking

→ **Figure 14:** Fault and error masking





Reliability

GPT: *"In the realm of digital circuits, reliability is the silent code that runs beneath the surface, orchestrating a symphony of zeros and ones with flawless harmony. Here, integrity is not just a trait but the very blueprint of innovation, ensuring every pulse and switch not only performs but endures."*

Reliability

- **Reliability** is one of the fundamental attributes of **dependable computing** and describes the ability of the system to deliver its intended service.
- Mathematically, reliability $R(t)$ at a time instance t represents the **probability of the system to perform without failure in the time range $[0, t]$** .
- Evidently, the reliability of the system is strongly related to the **fault tolerance designed** into the system and resulting application failure behavior
- An application's **failure rate** is correlated to the application-specific **tolerance to errors** and resulting deviation in application behavior.
- The type and extent of such tolerances may vary with the application domain and are characterized by **application-specific QoS requirements**.

Reliability

The fault tolerance specific QoS metrics can be categorized as follows:

→ **Functional Reliability**: With the rising constant **failure rates** during the normal life of the system, the chances of such failures manifesting as incorrect computations have also increased.

In applications that require **high levels of computational accuracy** such as financial transactions in point-of-sales systems or ATMs, scientific applications, the corresponding QoS can be expressed in terms of **functional reliability**.

It concerns the **correctness of the results computed by a system operating in a fault-inducing environment**.

The functional reliability can be quantified by the probability of no physical fault-induced errors occurring during application execution or the **Mean Time between Failures (MTBF)**.

Reliability

→ **Timing Reliability**: The performance of the system in terms of the expected behavior concerning the **timeliness** of execution completion can be expressed as its **timing reliability**.

It is used only in terms of **real-time systems** and depending upon the criticality of the application, can be expressed in terms of **Worst-Case Executing Time (WCET)**, **MTBF**, **Probability of Completion** and **Average Makespan**.

WCET is usually used in hard real-time systems such as pacemakers and automobile safety features where any instance of missing a deadline can have fatal consequences.

MTBF, frequently used in the context of **repairable systems**, can also be used for expressing the timing reliability in firm real-time systems such as manufacturing assembly lines, where infrequent failures can be tolerated, provided sufficient availability is ensured.

Average makespan and **probability of completion** are usually used in soft real-time systems such as streaming devices and gaming consoles where frequent deadline misses can be tolerated as long as they do not affect user experience.

Reliability

→ **Lifetime Reliability**: The expected operational life of the system can be characterized by its lifetime reliability.

Depending upon whether the system is **repairable** and the cost of such repairs, metrics such as **Mean Time To Detection (MTTD)**, **Mean Time To Failure (MTTF)**, **Mean Time To Crash (MTTC)**, and **Availability** can be used to characterize the system's lifetime reliability.

MTTF refers to the expected time to the first observed failure in the system. In healthcare applications and consumer electronics, the need for **predictable and extended MTTF** can be the primary objective.

MTTC refers to the expected operational time for the point at which the system does not have sufficient resources for ensuring the expected behavior and is usually applicable for repairable systems. In applications with long mission times such as space exploration, repairing the failure mechanism is used to extend the MTTC. However, repair time plays a critical role in high-availability applications such as automated control of power generation.

Reliability Estimation

→ Estimating the implicit fault-masking and the impact of additional fault mitigation measures forms an integral component of designing fault-tolerant system. While underestimation of the impact of faults can lead to unreliable system behavior, overly pessimistic estimations can result in overdesigning, eventually leading to infeasible designs in resource-constrained systems. Some of the more widely used reliability estimations are described below:

1. **Fault Injection (FI)** involves introducing faults/errors in a system to validate the device dependability under faulty conditions. FI can be **physical**, **stimulated**, or **emulated**.
2. **Analytical Reliability Estimation** usually involves using mathematical models and statistical data for the fast analysis of system behavior under faults. Some of the more widely used analytical estimation methods are: **Architectural Vulnerability Factor (AVF)**, **Probabilistic Transfer Matrix (PTM)**, **Timing and Lifetime Reliability**.

Fault Tolerance

- In order to achieve the **reliability-oriented QoS** requirements of a system, appropriate **fault tolerance measures** need to be implemented.
- Such measures can include **fault/error avoidance**, **detection** and **mitigation** for transient faults, **diagnosis** and **repair** for permanent faults, etc.
- These measures can be implemented at one/multiple layers of the computation stack.
- **Redundancy** forms a primary tool for implementing fault tolerance measures.
- Redundancy usually involves complementing the regular computation with additional processing in order to detect and/or mask the effect of errors during computation.

Fault Tolerance

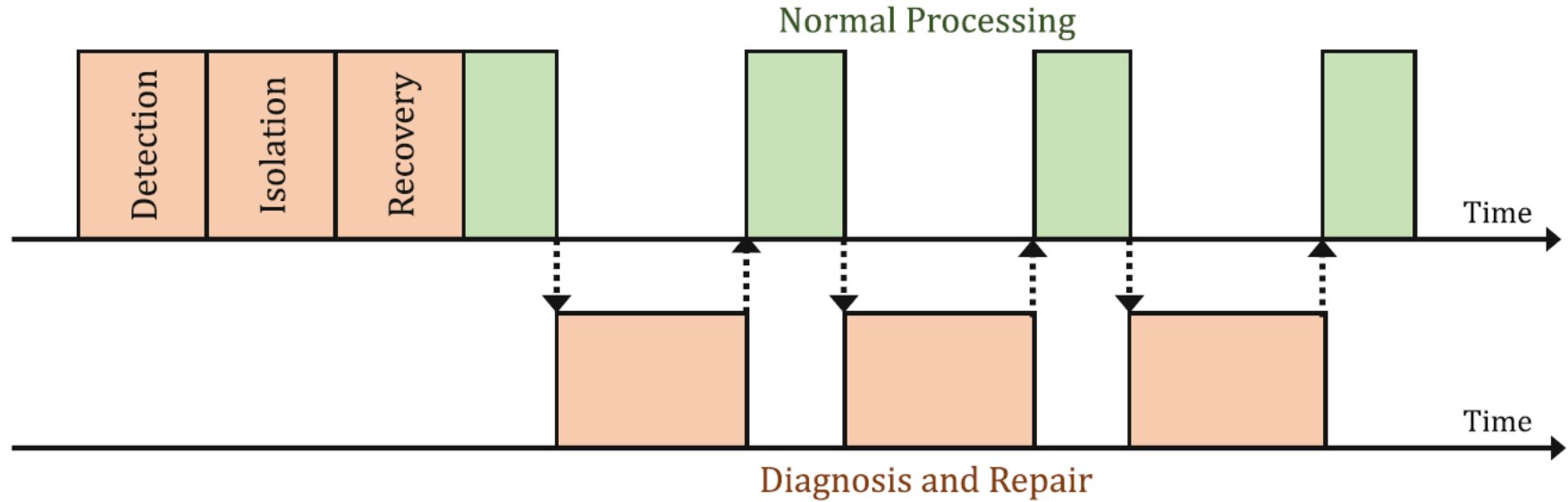
- The different activities involved in **fault tolerance** are shown in Figure 15. Depending upon the repairability of the system, implementing fault tolerance may involve a subset of the following:
- **Fault avoidance**: This usually involves building the system with robust components that are less susceptible to faults. It may also include avoiding system configurations that increase the susceptibility to fault mechanisms.
For **example**, given the impact of **Dynamic Voltage Scaling (DVS)** on **SER**, the designer may choose to avoid configurations with lower supply voltages in high-radiation environments.
 - **Detection**: This is primarily to ascertain whether the computation has been affected by any kind of faults. Depending upon the implementation, it may incur timing and/or resource overheads.
 - **Diagnosis**: Unlike detection, diagnosis concerns determining the fault characteristics. This includes finding the type of fault – transient or permanent, locating the component(s) causing the fault, estimating the impact of the fault on application behavior, etc.

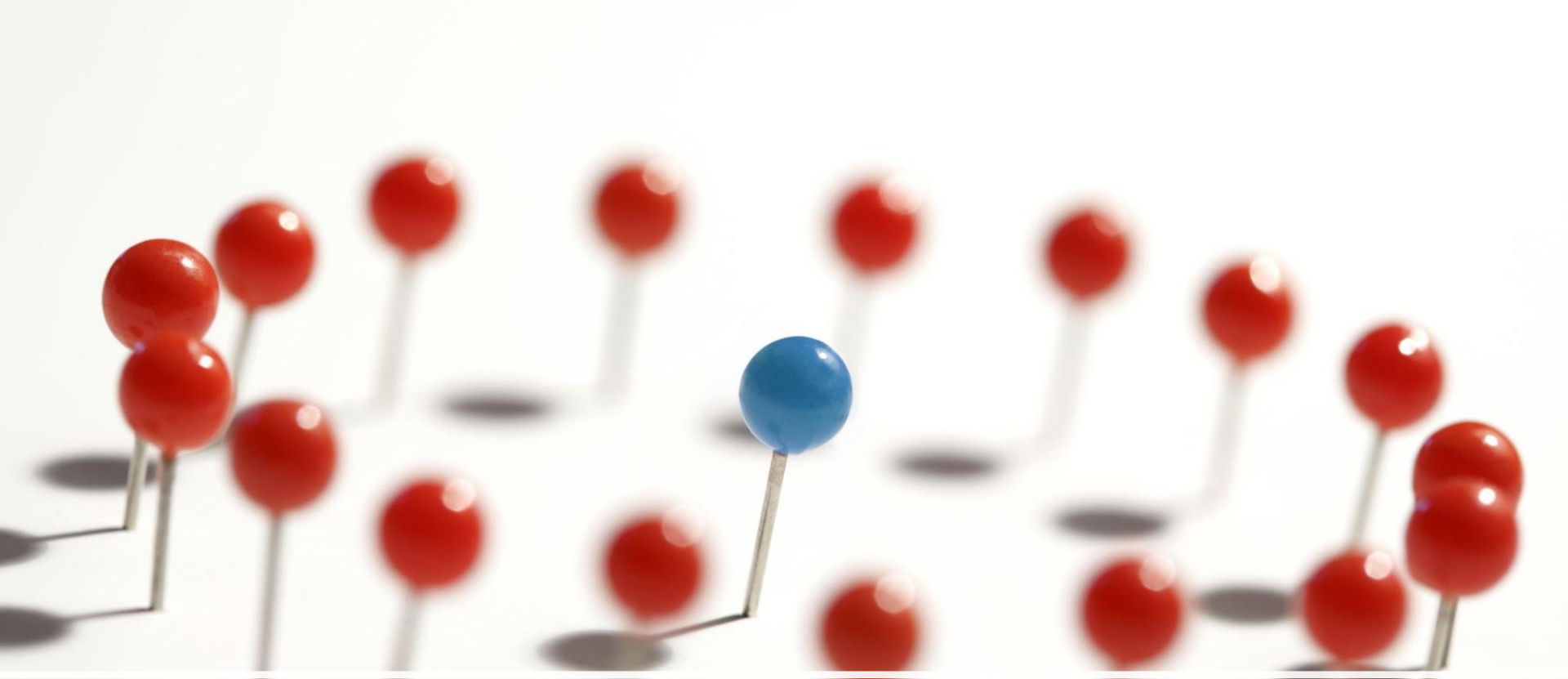
Fault Tolerance

- **Isolation:** This process forms an integral component of repairable systems and includes eliminating the faulty components from the computation until repair is completed.
In case of permanent faults, this may include finding the components (or sub-components) that need to be excluded from any future computation.
- **Repair:** In repairable systems, fault tolerance may include allowing the affected components to recover partially/completely to their operational states.
For instance, it may include reducing the workload on faster aging components to reduce thermal stress.
- **Recovery:** While repair usually concerns the components of the system, recovery primarily concerns the application execution.
It can range from the mitigation of any detected faults/errors to resetting the system from a hang state.

Fault Tolerance

→ **Figure 15:** Fault and error masking





Redundancy

Redundancy

- **Redundancy** can be defined as the **replication** of critical components and/or functions in a system with the primary goal of improving the system's reliability.
- For fault tolerance, different layers of the system may implement various forms of replication. Broadly, such measures can be categorized under three types of redundancies: **spatial/physical**, **temporal**, and **information**.
- Depending upon the extent to which each of these is implemented, it can result in varying **effectiveness** and **overheads**.

Redundancy

- Let us discuss each of the redundancy types in the context of a matrix multiplication and compare their impact on the functional, timing and lifetime reliability.
- Figure 16 shows the 4×4 matrix multiplication with the Multiply and Accumulate (MAC) considered as the computation module.
- Assuming the multiplication of two $N \times N$ matrices, the MAC would compute N^2 dot products.
- We consider the computation of each element of the output matrix as a single operation.
- Also, for the error model, we assume an average fault rate of λ following a Poisson distribution.
- Correspondingly, the probability of an error occurring with increasing execution time is depicted in Figure 17.

Redundancy

Figure 16: Matrix-matrix multiplication

$$\begin{bmatrix} c_{1,1} & c_{1,2} & c_{1,3} & c_{1,4} \\ c_{2,1} & c_{2,2} & c_{2,3} & c_{2,4} \\ c_{3,1} & c_{3,2} & c_{3,3} & c_{3,4} \\ c_{4,1} & c_{4,2} & c_{4,3} & c_{4,4} \end{bmatrix} = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} \\ a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} \\ a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} \\ a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} \end{bmatrix} \times \begin{bmatrix} b_{1,1} & b_{1,2} & b_{1,3} & b_{1,4} \\ b_{2,1} & b_{2,2} & b_{2,3} & b_{2,4} \\ b_{3,1} & b_{3,2} & b_{3,3} & b_{3,4} \\ b_{4,1} & b_{4,2} & b_{4,3} & b_{4,4} \end{bmatrix}$$

$C = AB$
 $A, B, C \in \mathbb{Z}^{N \times N}$

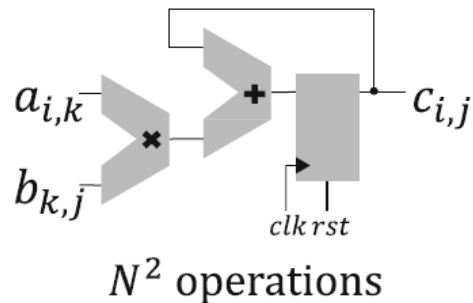
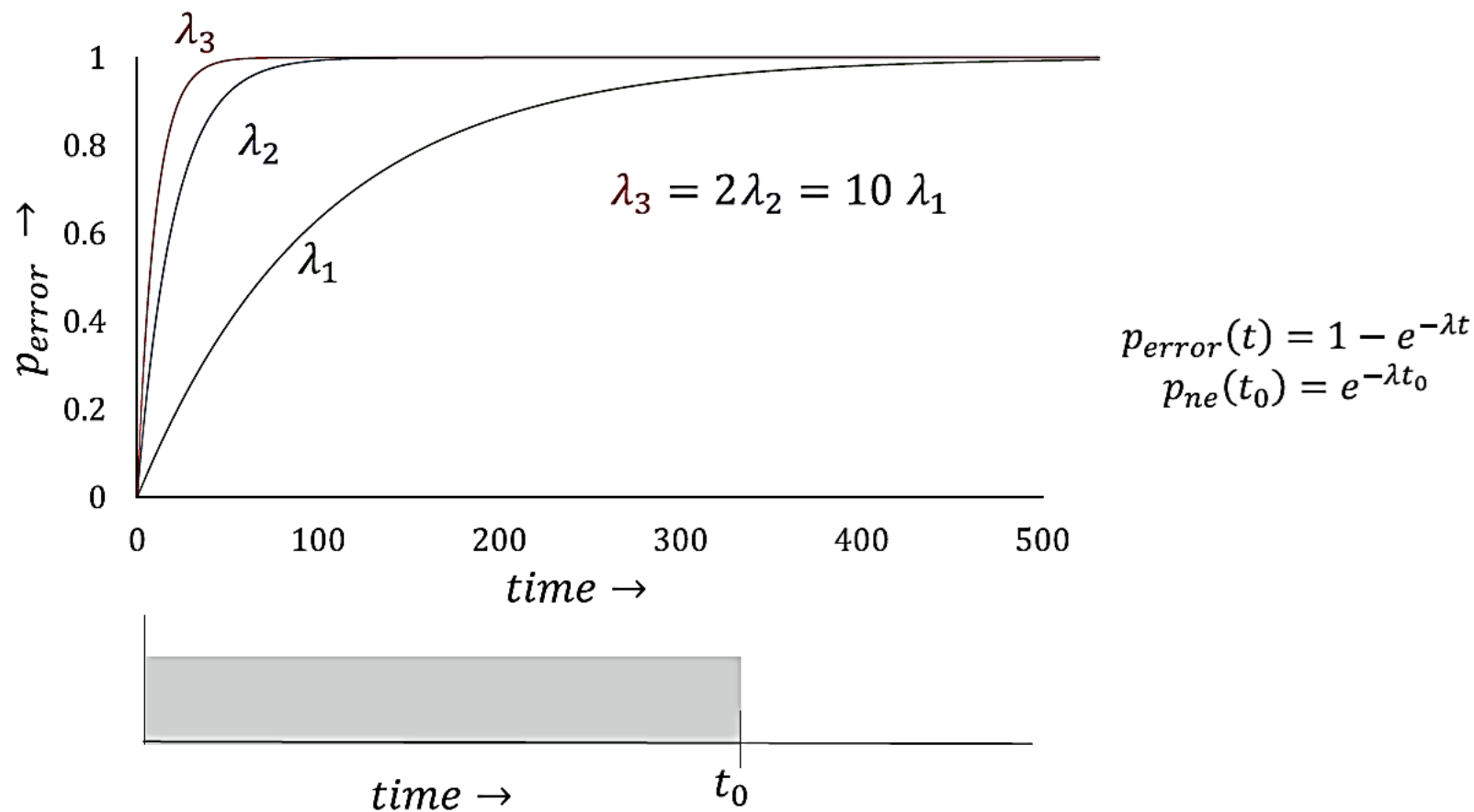


Figure 17: Error probability



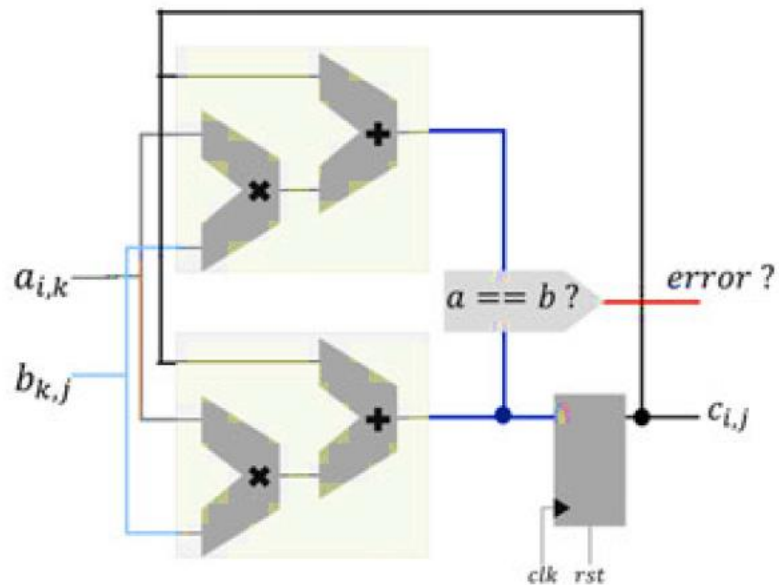
Spatial/Physical Redundancy

- **Spatial/Physical Redundancy** usually involves using additional hardware modules for replicating the computation.
- As shown in Figure 18.a, such replication could be targeted for detection of errors by using **Dual Modular Redundancy (DMR)**.
- Similarly, **Triple Modular Redundancy (TMR)** can be used to mask the effect of any errors in one of the modules, as shown in Figure 18.b.
- The **voting module** usually implements a majority function and can add an insignificant amount of timing overhead, as shown in Figure 21.
- Figure 18.c shows a more generalized version – **N -Modular Redundancy (NMR)**, where the result is considered acceptable if at-least **k -modules** provide the same output: a k -out-of- N : G system.

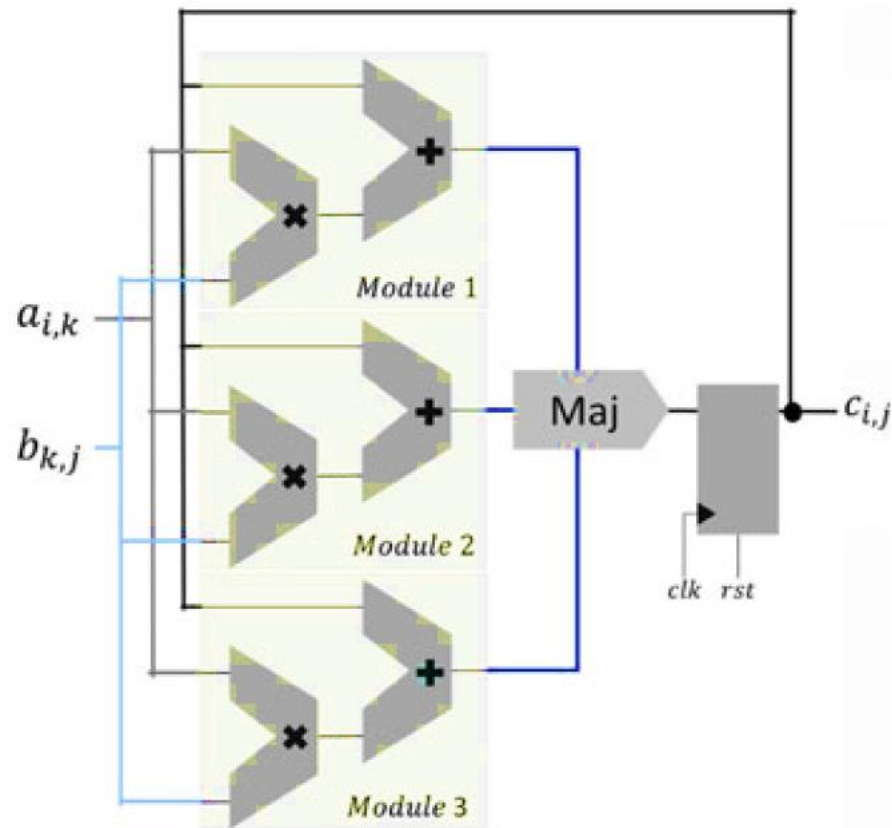
Spatial/Physical Redundancy

- For TMR (a 2-out-of-3:G system), the **probability of error** can be estimated as
$$3 \times p_{error}^2(t) \times p_{ne}(t) + p_{error}^3(t),$$
- which corresponds to the scenario where all three or any two of the three modules encounter an error during computation.
- It can be noted that the NMR methods shown in Figure 18 aim to improve the **functional reliability** but may adversely affect the **lifetime reliability** owing to the parallel execution and resulting higher thermal stress.
- **Spatial redundancy** for improving the lifetime reliability usually involves using **cold spares**.
- Here, **cold spares** refer to components or subsystems that are kept on standby and are not actively used in the system until they are needed to replace a failed component.
- Unlike **hot spares** or **warm spares**, cold spares are not powered on and do not operate in parallel with the active components.

Figure 18: Special Redundancy.



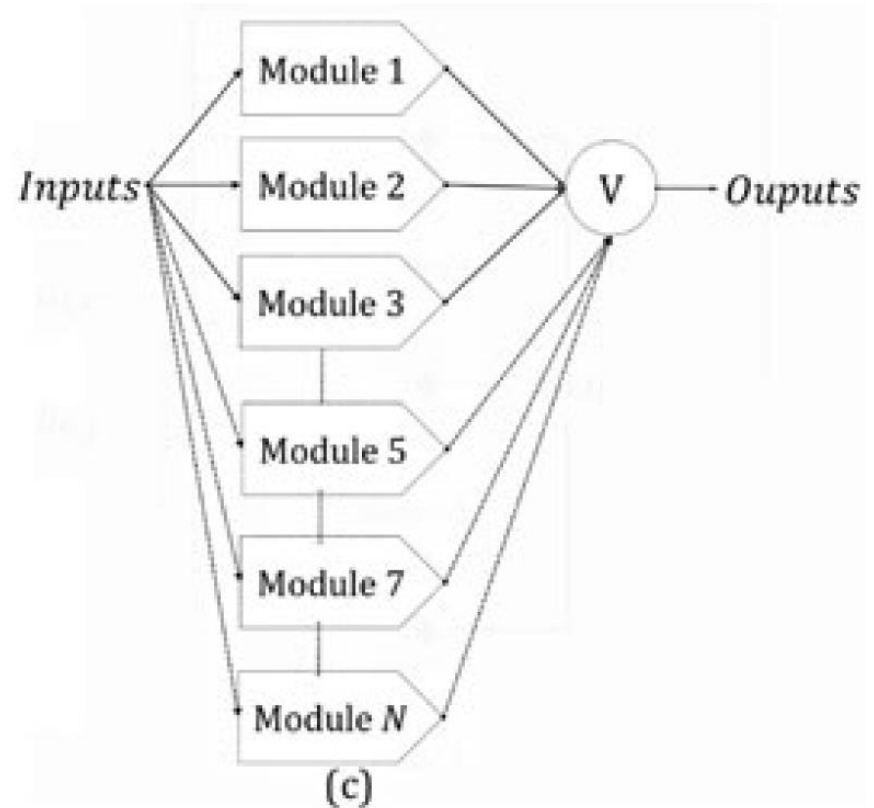
(a)



(b)

Redundancy

→ **Figure 18:** Special Redundancy.

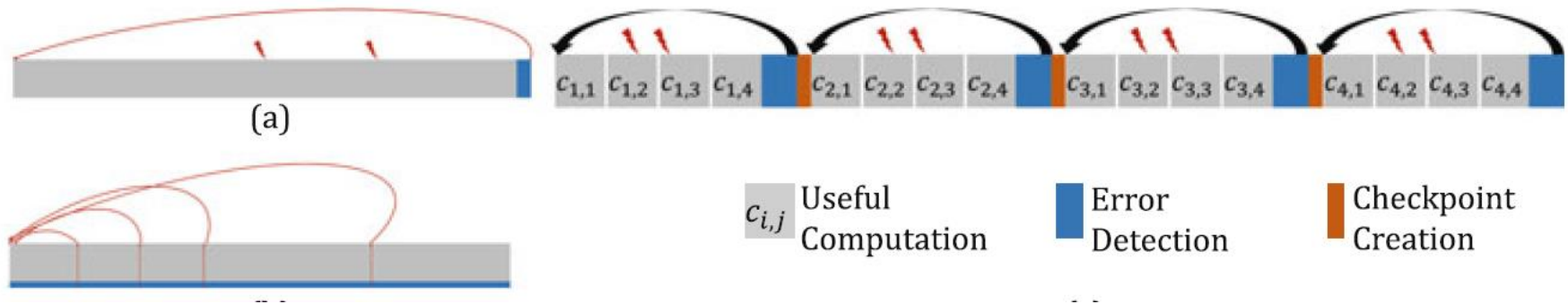


Temporal Redundancy

- **Temporal Redundancy** involves additional **re-execution** of the computation workload on the same hardware module.
- As shown in Figure 19.a, it may include **complete re-execution** when an error is detected at the end of the computation.
- Similarly, with parallel error detection, the re-execution may occur from **different points**, depending on the error occurrence, as shown in Figure 19.b.
- Figure 19.c shows the more widely used method of **check-pointing** with **roll back recovery**.
- In the context of the matrix multiplication, it may involve creating checkpoints after the computation of each row of the output matrix, such that any re-execution can begin by restoring the stored checkpoint.

Redundancy

→ **Figure 19:** Temporal Redundancy.



Temporal Redundancy

- The functional reliability of such methods is usually determined by the accuracy of the detection methods.
- As shown in Figure 20, the **timing reliability** of temporal redundancy-based methods requires more complex estimation approaches as the execution time is nondeterministic in such cases.
- Since **temporal redundancy** usually results in increased computation per unit workload, it eventually leads to a **lower operational lifetime**.

Redundancy

→ **Figure 20:** Information Redundancy.

$$\begin{bmatrix} c_{1,1} & c_{1,2} & c_{1,3} & c_{1,4} & c_{1,5} \\ c_{2,1} & c_{2,2} & c_{2,3} & c_{2,4} & c_{2,5} \\ c_{3,1} & c_{3,2} & c_{3,3} & c_{3,4} & c_{3,5} \\ c_{4,1} & c_{4,2} & c_{4,3} & c_{4,4} & c_{4,5} \\ c_{5,1} & c_{5,2} & c_{5,3} & c_{5,4} & c_{5,5} \end{bmatrix} = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} \\ a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} \\ a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} \\ a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} \\ a_{5,1} & a_{5,2} & a_{5,3} & a_{5,4} \end{bmatrix} \times \begin{bmatrix} b_{1,1} & b_{1,2} & b_{1,3} & b_{1,4} & b_{1,5} \\ b_{2,1} & b_{2,2} & b_{2,3} & b_{2,4} & b_{2,5} \\ b_{3,1} & b_{3,2} & b_{3,3} & b_{3,4} & b_{3,5} \\ b_{4,1} & b_{4,2} & b_{4,3} & b_{4,4} & b_{4,5} \end{bmatrix}$$

$$C_{fc} = A_{cc} \times B_{rc}$$

C_{fc} : $(N + 1)^2$ operations,
 + Error detection,
 + Error correction

$$a_{N+1,j} = \sum_{i=1}^N a_{i,j} \quad \text{for } 1 \leq j \leq N$$

$$b_{j,N+1} = \sum_{j=1}^N b_{i,j}, \quad \text{for } 1 \leq i \leq N$$

Information Redundancy

- **Information Redundancy** involves computation of additional data points in order to detect and/or recover any errors in computation.
- For instance, as shown in Figure 20, the input matrices are augmented with their column and row **checksums** resulting in larger operands Acc and Brc , respectively.
- Correspondingly, the output matrix is a full checksum version of the original output matrix: C_{fc} .
- As a result, it requires $(N + 1)^2$ operations instead of N^2 and, therefore, adversely affects both the timing and lifetime reliability.

Redundancy

- While **redundancy** forms a generic approach to providing **fault tolerance**, depending upon the component type, specialized methods are employed at one or more layers.
- Further, as shown in Figure 21, depending upon the type of redundancy, different levels of **timing overheads** are introduced into the execution.
- While **spatial** and **information redundancy** results in more **deterministic timing overheads**, **temporal redundancy** methods may lead to **nondeterministic overheads**, thereby requiring complex estimation methods.
- Methods for improving fault tolerance in computation under three categories:
 - **Single-core computing** includes all methods that can improve reliability of a single Processing Element (PE).
 - The methods that take advantage of the presence of **multiple PEs** on the architecture are described under **multicore computing**.
 - The **reconfigurable computing** methods are based on reconfigurable architectures such as FPGA and Coarse-Grained Reconfigurable Arrays (CGRA).

Redundancy

→ **Figure 21:** Redundancy and timing reliability.

